

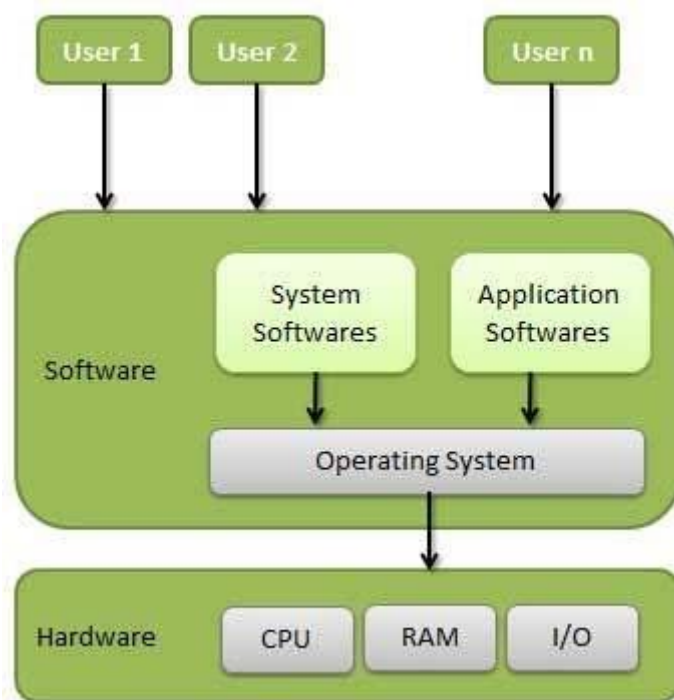
UNIT-1

INTRODUCTION TO OPERATING SYSTEMS

- 1.1 Define an operating system
- 1.2 Discuss history of operating system
- 1.3 Discuss about various types of operating systems
- 1.4 Distinguish spooling and buffering
- 1.5 Explain the concepts multiprogramming and timesharing
- 1.6 Differentiate between distributed and real time systems
- 1.7 Describe multiprocessor systems
- 1.8 Describe the operating system components
- 1.9 Discuss operating system services
- 1.10 Define system call with an example
- 1.11 List and explain different types of system calls
- 1.12 Define single user, multi user operating system structure

1.1 Define An Operating System

- An Operating System is system software that manages computer hardware and software resources and provides common services for computer programs.
- In simple terms, it acts as an interface between the user and the hardware, allowing you to interact with the computer without needing to understand the hardware's language.



Following are some of important functions of an operating System.

- Memory Management
 - Processor Management
 - Device Management
 - File Management
 - Security
 - Control over system performance
 - Job accounting
 - Error detecting aids
 - Coordination between other software and users
-
- **Memory Management:** Keeps track of each byte in a computer's memory and allocates space to programs when needed.
 - **Processor Management:** Assigns tasks to the CPU and manages how processes are executed.

- **Device Management:** Controls and coordinates input/output devices like keyboard, mouse, and printers.
- **File Management:** Organizes, stores, retrieves, and manages data in files on storage devices.
- **Security:** Protects data and system access from unauthorized users or threats.
- **Control over System Performance:** Monitors and improves the speed and efficiency of the system.
- **Job Accounting:** Tracks system usage by users or tasks for billing or monitoring.
- **Error Detecting Aids:** Identifies and reports errors to ensure smooth functioning.
- **Coordination Between Software and Users:** Helps software and users interact effectively with the hardware.

1.2 Discuss History Of Operating System

The first operating system was introduced in 1956. It was a Batch Processing Systems GM-NAA I/O (1956) that automated job handling.

1. Early Systems (1940s–1950s) – No OS

- Computers were huge and expensive.
- Programs were entered using switches and punch cards.
- There was no operating system; users interacted directly with hardware.

2. Batch Processing Systems (1950s–1960s)

- Jobs (programs) were collected and run in batches without user interaction.
- An early form of OS managed job execution and switching.
- Examples: IBM's early batch systems.

3. Multiprogramming and Time-Sharing (1960s–1970s)

- OS began supporting multiple programs in memory at once.
- Time-sharing systems allowed many users to use the computer at the same time.
- Examples: UNIX (developed in the 1970s), MULTICS.

4. Personal Computer Operating Systems (1980s)

- Rise of personal computers led to OS like MS-DOS, Mac OS, and early Windows.
- User-friendly interfaces (GUIs) started to develop.
- OS supported applications, file systems, and basic multitasking.

5. Modern Operating Systems (1990s– 2000s)

- Full multitasking, multi-user, networking, and security features.
- OS now runs on desktops, laptops, servers, and mobile devices.
- Examples: Windows, macOS, Linux, Android, iOS.

6. AI Integration – (Ongoing) With the growth of time, Artificial intelligence came into picture. Operating system integrates features of AI technology like Siri, Google Assistant, and Alexa and became more powerful and efficient in many way.

1.3 Various Types Of Operating Systems

Types of Operating Systems

- Batch Operating System
- Distributed Operating System
- Multitasking / Time-Sharing Operating System
- Network Operating System
- Real-Time Operating System (RTOS)
- Mobile Operating System
- Multiprocessor Operating System

1. Batch Operating System

A Batch Operating System processes a group of similar jobs together without user interaction. Users prepare jobs offline and submit them to the computer operator. The system executes jobs in the order they arrive. It works by automatically queuing and running one job after another without manual intervention.

A real-time example is IBM's early batch processing systems like SCOPE.

2. Distributed Operating System

A Distributed Operating System manages a group of independent computers and makes them function as a single system. It shares resources, data, and tasks across connected systems to improve efficiency and performance. This type of OS is useful in networks that need coordination between multiple systems. It works by linking multiple machines to work together, distributing tasks and resources.

Examples include LOCUS, Amoeba, and Google's internal system, Borg.

3. Multitasking / Time-Sharing Operating System

A Multitasking or Time-Sharing Operating System allows many users or processes to use the computer simultaneously. The CPU switches between tasks rapidly, giving the impression of parallel execution. This improves responsiveness and resource utilization. It works by dividing CPU time into small slices and switching between tasks quickly.

Examples include UNIX, Linux, and Windows 10.

4. Network Operating System

A Network Operating System supports systems that are connected via a network. It enables file sharing, printer access, and communication between devices. Common in

client-server environments, it manages network resources efficiently. It works by handling data and resource sharing across devices over a network.

Examples include Microsoft Windows Server and Novell NetWare.

5. Real-Time Operating System (RTOS)

A Real-Time Operating System (RTOS) is designed to process data and events within strict time limits. It is used in systems where quick responses are crucial, such as in embedded systems or industrial controls. RTOS ensures reliable and timely execution. It works by prioritizing tasks and executing them within guaranteed time frames.

Real-time examples include VxWorks, FreeRTOS, and QNX.

6. Mobile Operating System

A Mobile Operating System is specially designed for mobile devices like smartphones and tablets. It supports touch input, mobile apps, wireless connectivity, and efficient power management. This OS is lightweight and optimized for mobile hardware. It works by managing mobile resources and running applications smoothly.

Examples include Android and iOS.

7. Multiprocessor Operating System

A Multiprocessor Operating System is used in systems with two or more CPUs. It increases speed and reliability by allowing parallel processing of tasks. Such systems can share workloads and provide fault tolerance. It works by coordinating multiple processors to execute tasks simultaneously.

Examples include Linux with SMP support and Windows Server.

1.4 Spooling And Buffering

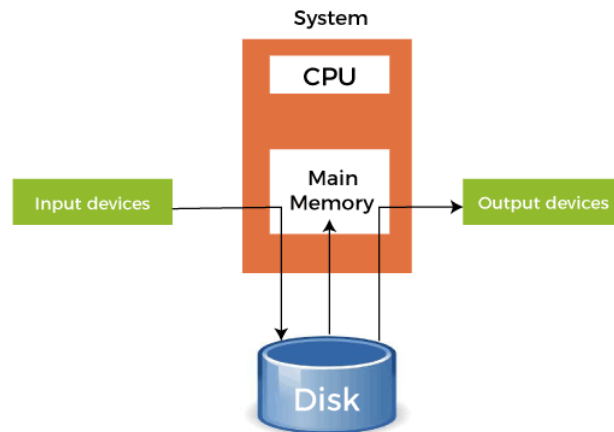
What is Spooling?

Spooling stands for Simultaneous Peripheral Operations online. It is the mechanism of putting data of various Input-Output (IO) jobs in a buffer. In other words, it uses a buffer to hold jobs for a device until it is ready to accept the job. A buffer is an area in main memory that holds data during input and output data transfers. In spooling, it is possible to read input of one job while printing the output of another job.

How Spooling Works in Operating System ?

1. Spooling involves creating a buffer called SPOOL, which is used to hold off jobs and data till the device in which the SPOOL is created is ready to make use and execute that job or operate on the data.

2. When a faster device sends data to a slower device to perform some operation, it uses any secondary memory attached as a SPOOL buffer. This data is kept in the SPOOL until the slower device is ready to operate on this data. When the slower device is ready, then the data in the SPOOL is loaded onto the main memory for the required operations.



Example of Spooling

- A student presses "Print" to print a 10-page assignment.
- The print job is not sent directly to the printer.
- Instead, it is stored in a spool (queue) on the hard disk.
- The printer takes jobs one by one from the spool.
- While printing, the student can continue using the computer.
- The OS handles other print jobs in the order they arrived.
- All jobs are stored in a queue and printed one by one, in the order they came (FIFO - First In First Out).

Advantages of Spooling:

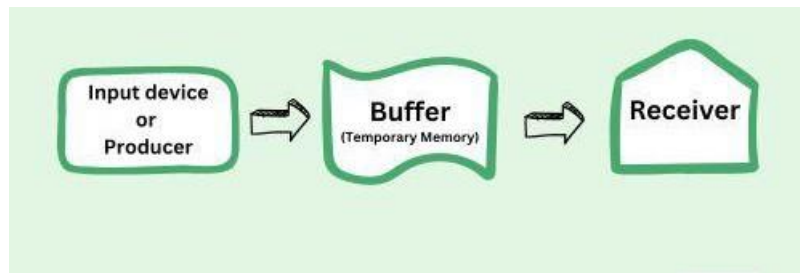
- Allows multiple print or I/O jobs to be queued.
- Improves CPU utilization by running jobs in the background.
- Supports multiple users sending tasks at the same time.

Disadvantages of Spooling:

- Requires additional storage space for the spool queue.
- Jobs may get delayed if the queue is too long.
- If spool file is corrupted, jobs may be lost.

Buffering in Operating System

The buffer is an area in the main memory used to store or hold the data temporarily. In other words, buffer temporarily stores data transmitted from one place to another, either between two devices or an application. The act of storing data temporarily in the buffer is called buffering.



Real-Life Examples:

1. Video Streaming:

Videos are loaded a few seconds ahead into a **buffer** so playback doesn't stop if internet speed drops.

2. Keyboard Input:

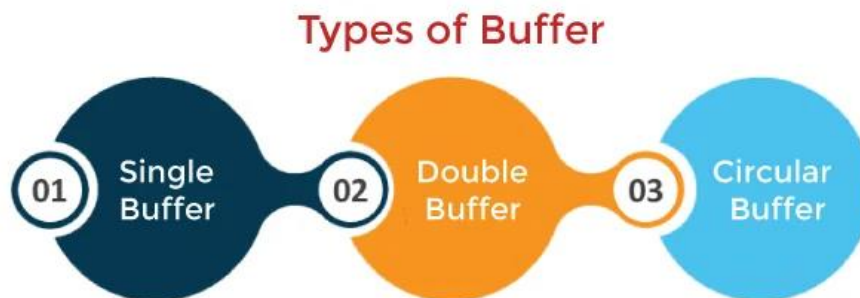
When you type fast, the keystrokes go into a **keyboard buffer** so the computer can process them one by one.

3. Printer:

Data is sent to a **print buffer** so the printer can print line by line while the computer continues its work.

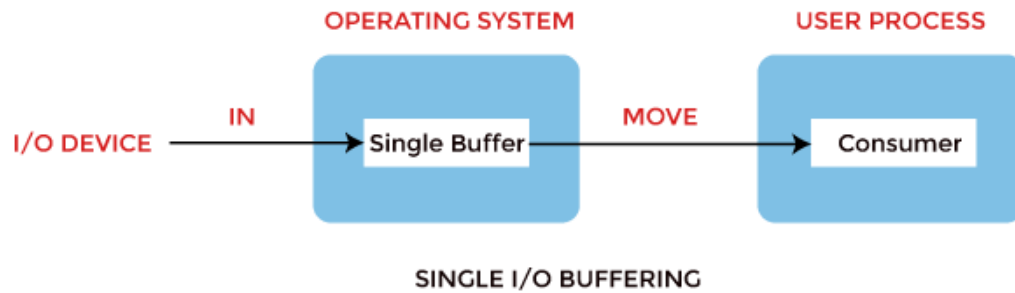
Types of Buffering

There are three main types of buffering in the operating system, such as:



1. Single Buffer

In Single Buffering, only one buffer is used to transfer the data between two devices. The producer produces one block of data into the buffer. After that, the consumer consumes the buffer. Only when the buffer is empty, the processor again produces the data.



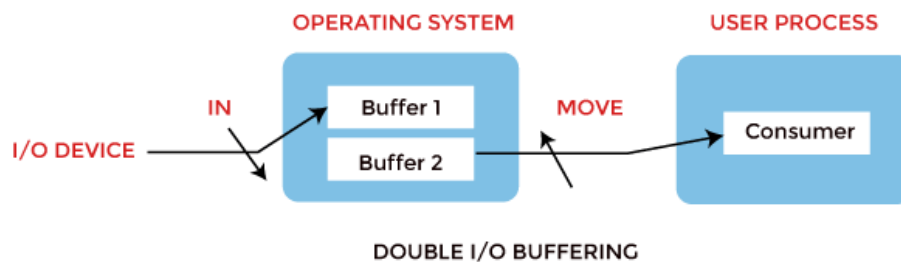
Real-life example:

Typing on a keyboard — the characters go into a single buffer before appearing on screen.

Limitation: Slower, since CPU and I/O can't work in parallel.

2. Double Buffering

In Double Buffering, two schemes or two buffers are used in the place of one. In this buffering, the producer produces one buffer while the consumer consumes another buffer simultaneously. So, the producer not needs to wait for filling the buffer. Double buffering is also known as buffer swapping.



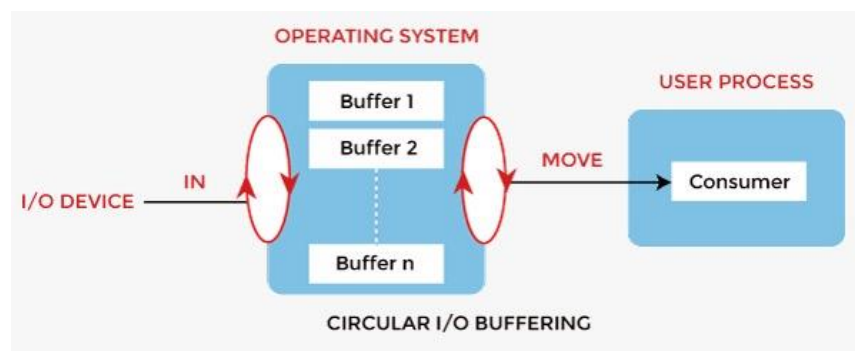
Real-life example:

Watching a video — while one part of the video plays from one buffer, the next part is loading into the second buffer.

Benefit: Faster than single buffering.

3. Circular Buffer (Ring Buffer)

When more than two buffers are used, the buffers' collection is called a circular buffer. A buffer is being one unit in the circular buffer. The data transfer rate will increase using the circular buffer rather than the double buffering.



Real-life example:

Audio recording — data is captured continuously, and the circular buffer ensures there's always space to store incoming data without waiting.

Benefit: Efficient for continuous and repeated I/O operations.

Advantages of Buffering

1. Improves overall system speed by reducing CPU waiting time.
2. Allows smooth data transfer between fast and slow devices.
3. Prevents data loss during input/output operations.

Disadvantages of Buffering

1. Uses extra memory (RAM), which may not be available in limited systems.
2. Poorly managed buffers can lead to delays or system lag.
3. Increases system complexity in buffer allocation and management.

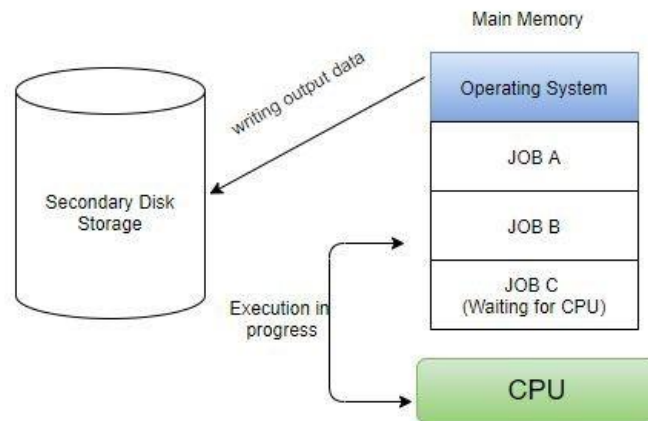
Difference between Spooling and Buffering

Point	Spooling	Buffering
1. Definition	Uses a disk to temporarily store data for slow devices	Uses RAM to temporarily store data during transfer
2. Speed	Works well with very slow devices (e.g., printers)	Manages speed differences between sender & receiver
3. Storage Location	Data is stored on hard disk (spool area)	Data is stored in main memory (RAM)
4. Multitasking	Allows multiple jobs to be queued and scheduled	Handles single data flow efficiently
5. Real-Life Example	Printing multiple files in a queue	Watching a YouTube video without pause

1.5 Multiprogramming Operating System

A Multiprogramming Operating System is designed to allow multiple programs to be loaded into memory and run simultaneously, improving CPU efficiency. In this system, while one program is waiting for input/output (I/O) operations (like reading from a disk or printing), the CPU switches to another program that is ready to use the processor. This way, the CPU is never idle and always working on something, increasing the overall speed and performance of the system. Multiprogramming makes better use of system resources and

supports multiple users or tasks at the same time, though only one program is actually executed at any given instant. It forms the foundation for more advanced systems like multitasking and time-sharing.



- Multiple programs are loaded into memory.
- The CPU picks one program and starts running it.
- If that program needs to wait (e.g., for a file to load), the CPU switches to another program.
- This continues in a cycle, increasing CPU efficiency

Advantages of Multiprogramming Operating System

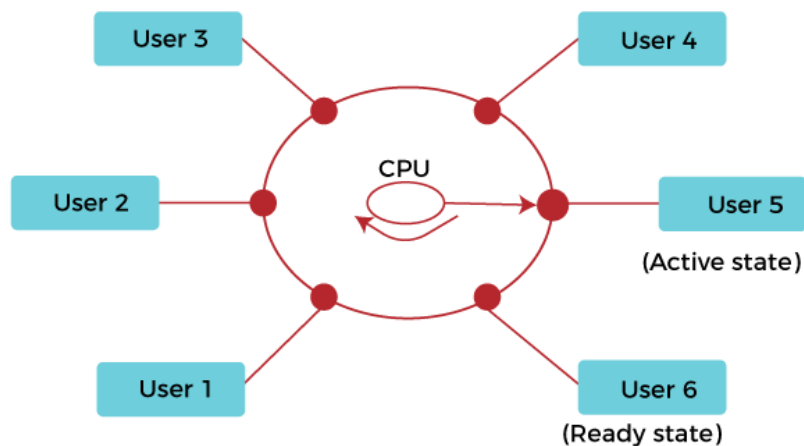
1. Increases CPU utilization by keeping it busy at all times.
2. Allows multiple programs to run, improving system efficiency.
3. Reduces idle time by switching tasks during I/O waits.

Disadvantages of Multiprogramming Operating System

1. Complex memory management is required to handle many programs.
2. Needs proper CPU scheduling to avoid conflicts.
3. Risk of one faulty program affecting others.

Time-Sharing Operating System

A Time-Sharing Operating System allows multiple users or programs to use a computer system at the same time by sharing the CPU time. It works by dividing CPU time into small time slices, and each user or task gets a turn to use the processor for a short period. This switching happens so quickly that all users feel like their programs are running continuously. Time-sharing improves response time, supports interactive computing, and is widely used in systems where many users access the same computer simultaneously, such as in universities or online servers. It is an extension of multiprogramming and plays a key role in multi-user environments.



1. Users 1 to 6 are all connected to the system and want to use the CPU.
2. The CPU allocates a small time slice to each user in a rotating (circular) manner this is shown by the circular arrow in the image.
3. At each step, the CPU switches to the next user after their time slice is over ensuring fair access to everyone.
4. User 5 is currently in the Active State (getting CPU time) and User 6 is in the Ready State, waiting for the CPU.
5. Because switching happens very fast, all users feel like their programs are running simultaneously, even though the CPU handles them one at a time.

Advantages of Time-Sharing Operating System (5 Lines)

1. Allows multiple users to use the system at the same time.
2. Provides quick response time for interactive tasks.
3. Maximizes CPU utilization by switching between users efficiently.

Disadvantages of Time-Sharing Operating System (5 Lines)

1. Requires complex CPU scheduling and resource management.
2. Too many users can slow down the system.
3. High chance of **data security** and **privacy issues** among users.

1.5 Distributed Operating System

A distributed operating system (DOS) is an essential type of operating system. Distributed systems use many central processors to serve multiple real-time applications and users. As a result, data processing jobs are distributed between the processors.



It connects multiple computers via a single communication channel. Furthermore, each of these systems has its own processor and memory. Additionally, these CPUs communicate via high-speed buses or telephone lines. Individual systems that communicate via a single channel are regarded as a single entity. They're also known as loosely coupled systems.

Types of Distributed Operating System

There are various types of Distributed Operating systems. Some of them are as follows

- Client-Server Systems
- Peer-to-Peer Systems
- Middleware
- Three-tier
- N-tier

Client-Server Systems

A model where **clients request services** and **servers provide** those services. The server handles tasks like database access, file storage, or web hosting. It is commonly used in networks like websites and banking systems.

Peer-to-Peer (P2P) Systems

In P2P, each computer (peer) can act as both client and server. All peers share resources directly without a central server. Used in file sharing systems like BitTorrent.

Middleware

Middleware is a software layer between the operating system and applications. It helps different systems or apps communicate and work together.

Three-Tier Architecture

This model has three layers: Presentation (UI), Logic (processing), and Data (database). Each layer works independently, improving performance and maintenance. Used in web apps and enterprise systems.

N-Tier Architecture

An extension of three-tier with more than three layers (e.g., security, business rules). Each tier can run on different servers for better scalability and flexibility. Used in large, complex applications like e-commerce or ERP systems

Advantages of Distributed Operating System

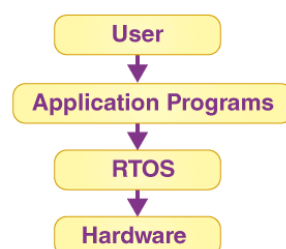
1. Resource Sharing – Users can share files, printers, and applications easily across systems.
2. Improved Performance – Tasks are divided among multiple computers, speeding up processing.
3. Fault Tolerance – If one system fails, others can take over its tasks.

Disadvantages of Distributed Operating System

1. Network Dependency – The system heavily depends on network connections; if the network fails, the system may not work properly.
2. Security Risks – More systems connected means higher chances of unauthorized access or data theft.
3. Complexity – Designing and managing a distributed system is more difficult than a single system.

Real-Time operating system

A Real-Time Operating System (RTOS) is a type of operating system designed to process data and provide results within a guaranteed time limit. It is mainly used in systems where timing is critical, such as in medical devices, industrial control systems, robots, and airbag systems in cars. RTOS responds to inputs immediately or within a fixed time, ensuring smooth and predictable operation. It supports real-time task scheduling, quick switching between tasks, and high reliability. There are two types: Hard real-time systems, where missing a deadline can cause failure, and Soft real-time systems, where occasional delays are acceptable.

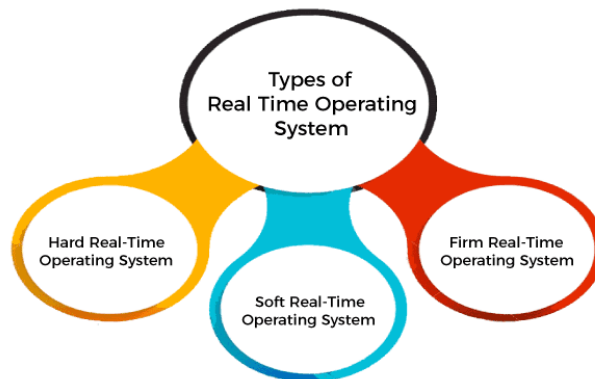


Applications of Real-Time Operating System

1. Airbag control systems in cars, where instant response is required during collisions.
2. Medical devices like pacemakers and patient monitors that need accurate timing.
3. Industrial automation systems that control machines and robots in factories.
4. Flight control systems in airplanes for real-time navigation and stability.
5. Multimedia systems for audio/video streaming where delay can affect performance

Types of Real-time operating system

Following are the three types of RTOS systems are:



1. Hard Real-Time Operating System

- Must respond **within a strict time limit**.
- Even a small delay can cause **failure**.

Used in:

- Medical devices
- Airplane control systems
- Industrial robots

2. Soft Real-Time Operating System

- Tries to meet timing needs, but **small delays are acceptable**.
- Not as strict as hard real-time.

Used in:

- Online video streaming
- Voice calls
- Multimedia systems

3. Firm Real-Time Operating System *(less common, between hard and soft)*

- Missing a deadline is **rarely acceptable**, but not as dangerous as hard real-time.
- The result becomes **useless** if delayed, but not harmful.

Used in:

- Banking systems (e.g., stock trading platforms)
- High-speed data logging

Advantages of Real-Time Operating System

1. Provides fast and timely responses to tasks.
2. Ensures high reliability and stability in critical systems.
3. Offers efficient memory and task management.

Disadvantages of Real-Time Operating System

1. Difficult to design and program due to timing constraints.
2. Requires special hardware for precise timing.

3. Limited user interface and features compared to general OS.